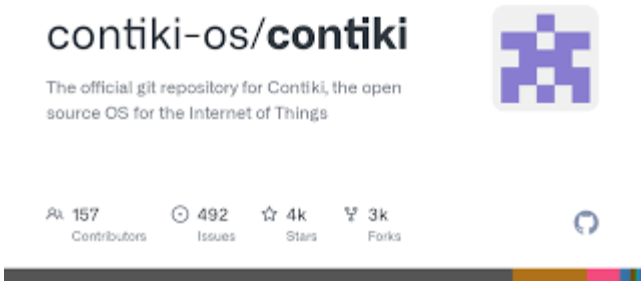
Aim: Installation process of Contiki OS and Cooja Simulator

Objectives

- To understand Contiki OS and its applications in the Internet of Things (IoT).
- To install the required software dependencies and development tools.
- To download and configure the Contiki-NG operating system.
- To build and launch the Cooja Simulator.

Introduction

Contiki OS is a lightweight operating system designed for resource-constrained embedded devices, wireless sensor networks, and Internet of Things applications. It provides networking capabilities and supports the development of applications for devices with limited memory and processing power. Contiki-NG is the actively maintained successor to the original Contiki OS project.



Cooja is a network simulator associated with Contiki-NG. It allows users to create virtual sensor nodes, execute embedded applications, and observe network behavior without requiring physical hardware. It provides a graphical interface for configuring simulations and examining communication between nodes.

In this experiment, Contiki-NG and Cooja were installed in Ubuntu 24.04 LTS running through WSL2 on Windows. Java Development Kit (JDK) 25 and Gradle were used to prepare and launch the simulator.

Theory

Contiki-NG

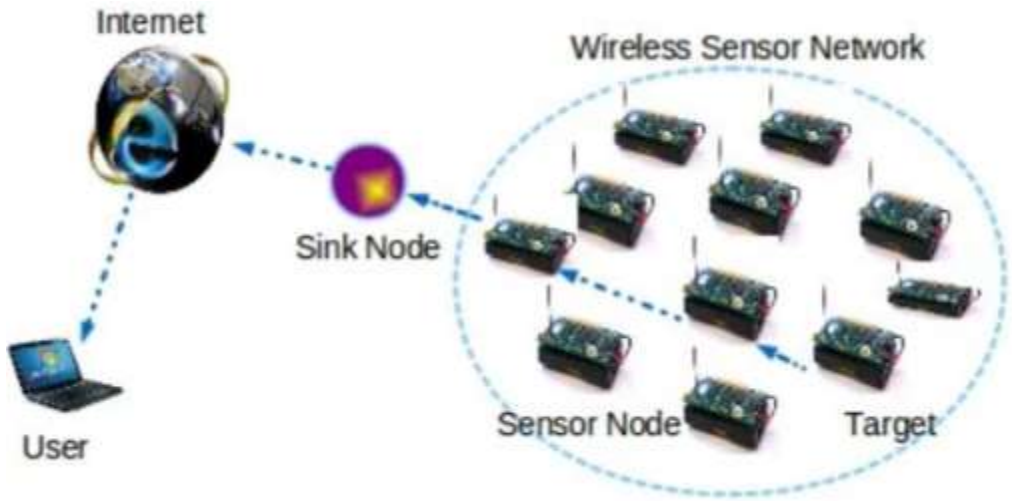
Contiki-NG is an open-source operating system designed for low-power IoT devices and wireless sensor networks. It supports embedded networking protocols and provides a development environment for writing and compiling applications for constrained devices.

Cooja Simulator

Cooja is a graphical network simulator used to develop and test wireless sensor network applications. It allows the user to create virtual sensor nodes, configure their properties, execute applications, and observe network behavior.

Wireless Sensor Network

A Wireless Sensor Network (WSN) consists of multiple sensor nodes that communicate wirelessly to collect or exchange information. Each node can perform sensing, processing, and communication tasks. Network simulators such as Cooja help evaluate network behavior before deployment on physical hardware.



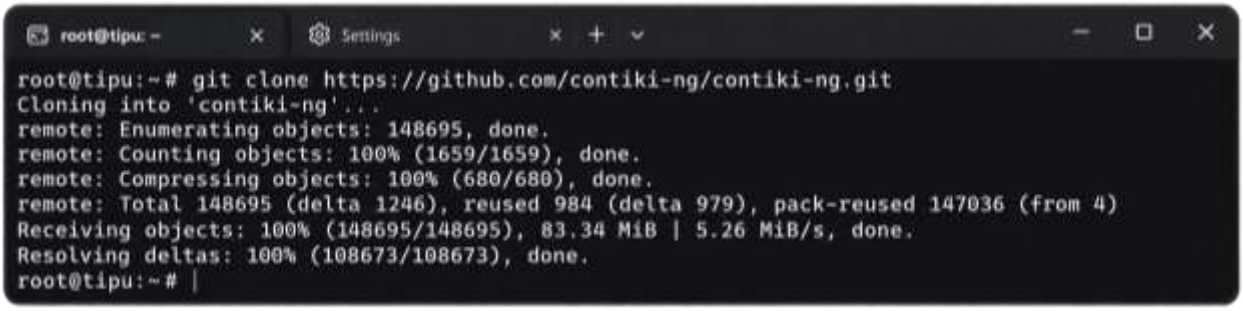
Setup

```
Ubuntu was opened through WSL2, and the following commands were executed:
Isb release -a
uname -m
whoami
The system reported Ubuntu 24.04.1 LTS, a 64-bit x86 architecture, and the root user.
The graphical display environment was checked using:
echo "SDISPLAY"

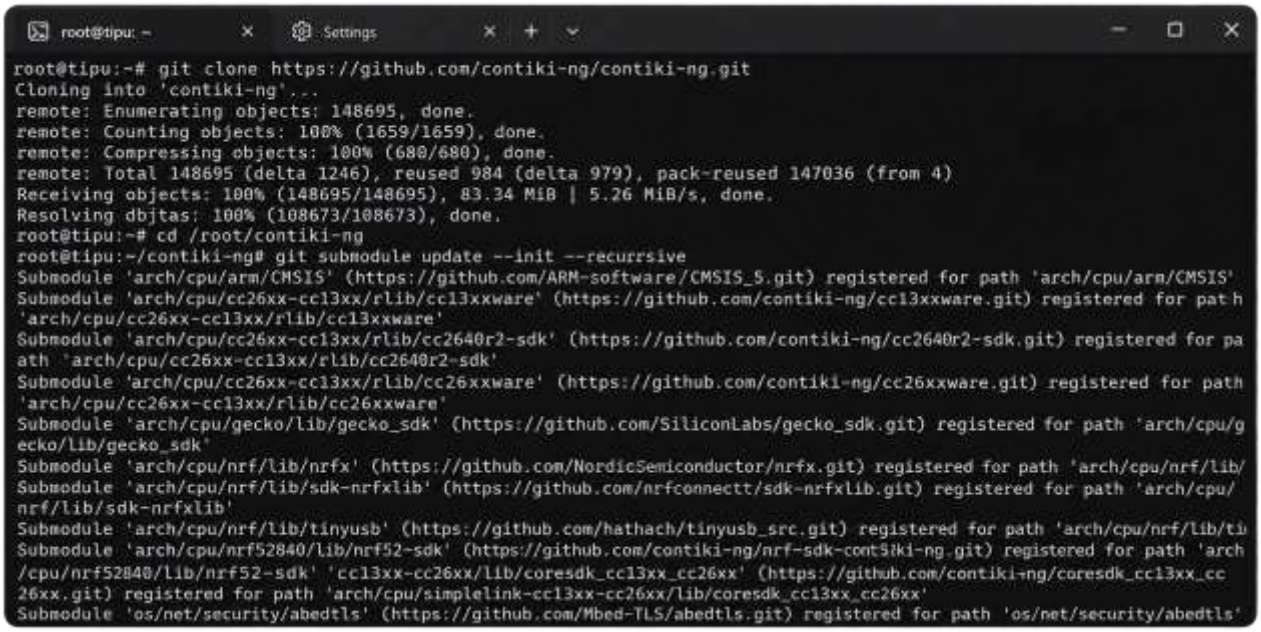
The Ubuntu package lists were updated using: apt update This command refreshes the package
information available from the configured Ubuntu repositories.

The development tools were installed using:
apt install -y git curl wget unzip ant default-jdk \
build-essential make gcc g++ python3 libncurses-dev
These packages provide Git for downloading source code, Java for running Cooja, Ant for
building legacy projects if needed, GCC and G++ for compilation, and other supporting
utilities.

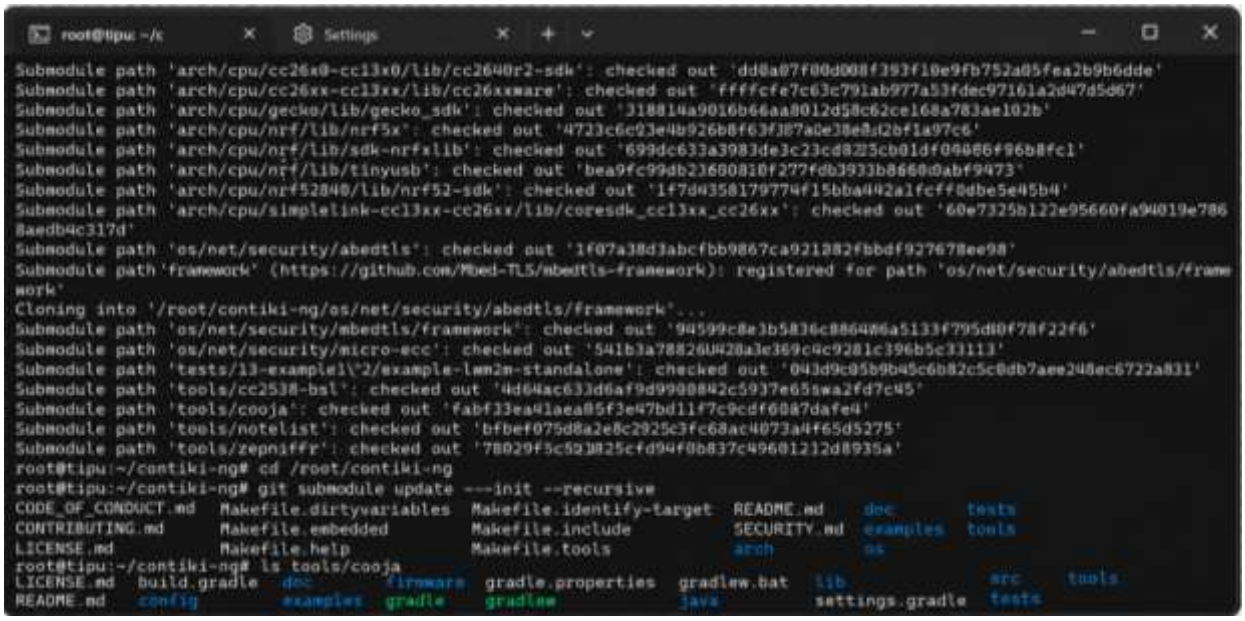
The Contiki-NG repository was downloaded using:
cd /root
git clone https://github.com/contiki-ng/contiki-ng.git
```



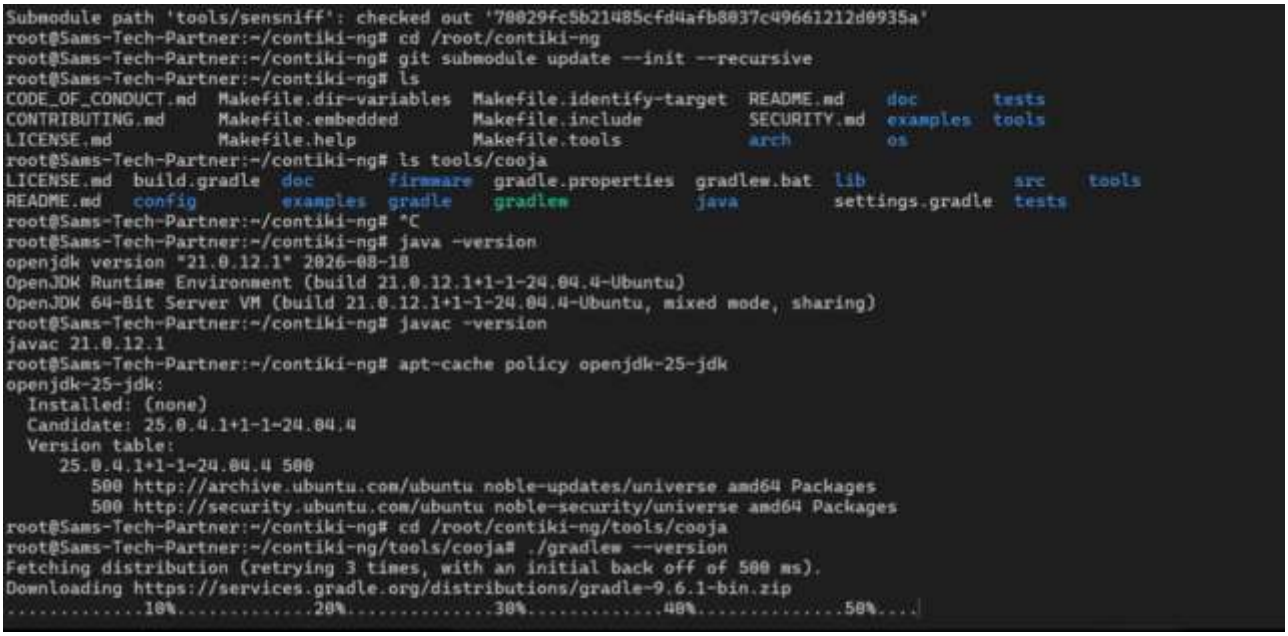
The downloaded repository was opened, and its submodules were initialized using:
cd /root/contiki-ng
git submodule update --init -recursive



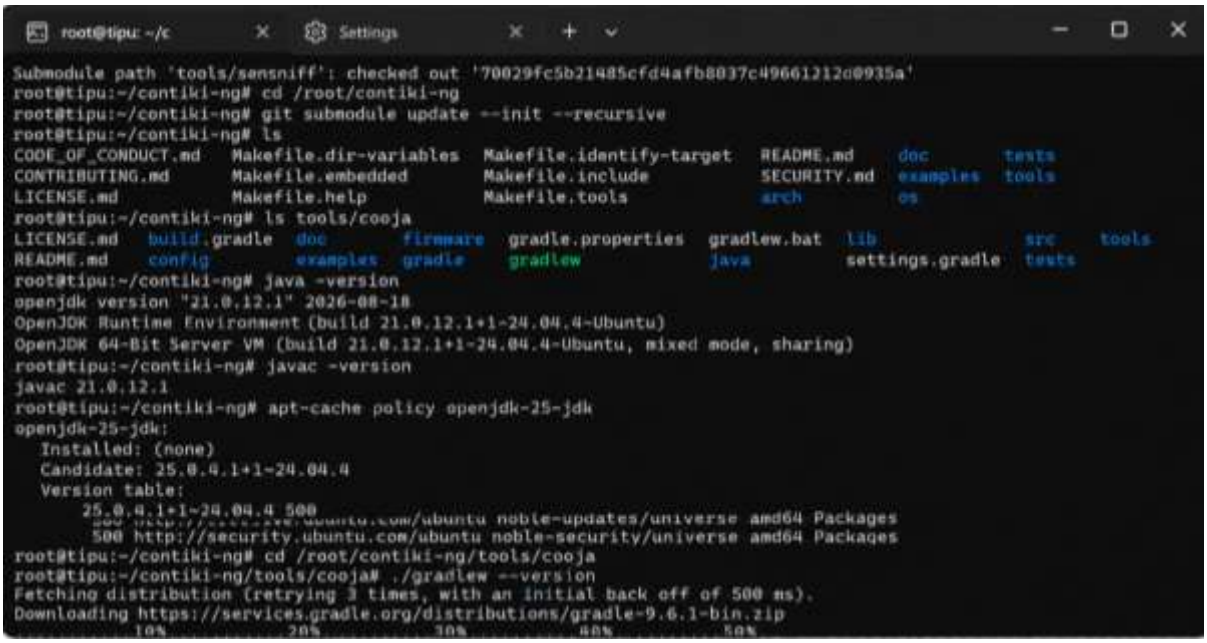
This command retrieves the supporting components referenced by the repository.
The downloaded files were verified using:
Is tools/cooja



The presence of directories such as examples, arch, os, and tools, along with Cooja files such as build.gradle and gradlew, confirmed that the required source files were present.
step 6: Install and Verify Java JDK 25
The installed Java version was initially checked using:
java -version
javac -version
Java 21 was initially available. JDK 25 was then installed because it matches the requirements of the current Contiki-NG development version.
apt install -y openjdk-25-jdk
The active Java version was verified again:
java -version
javac -version

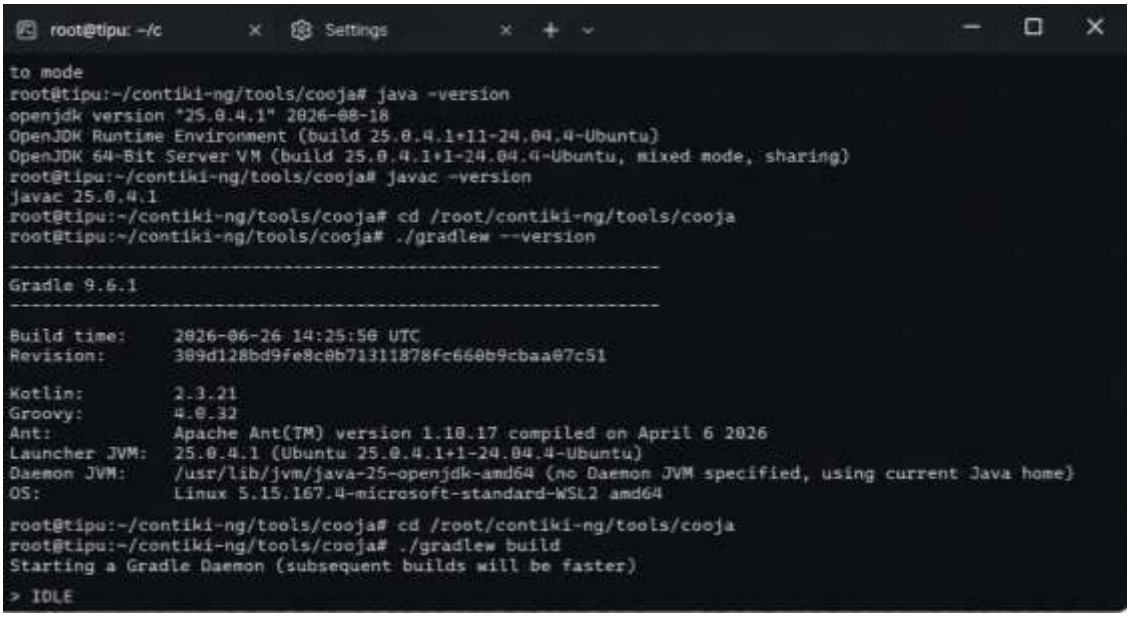


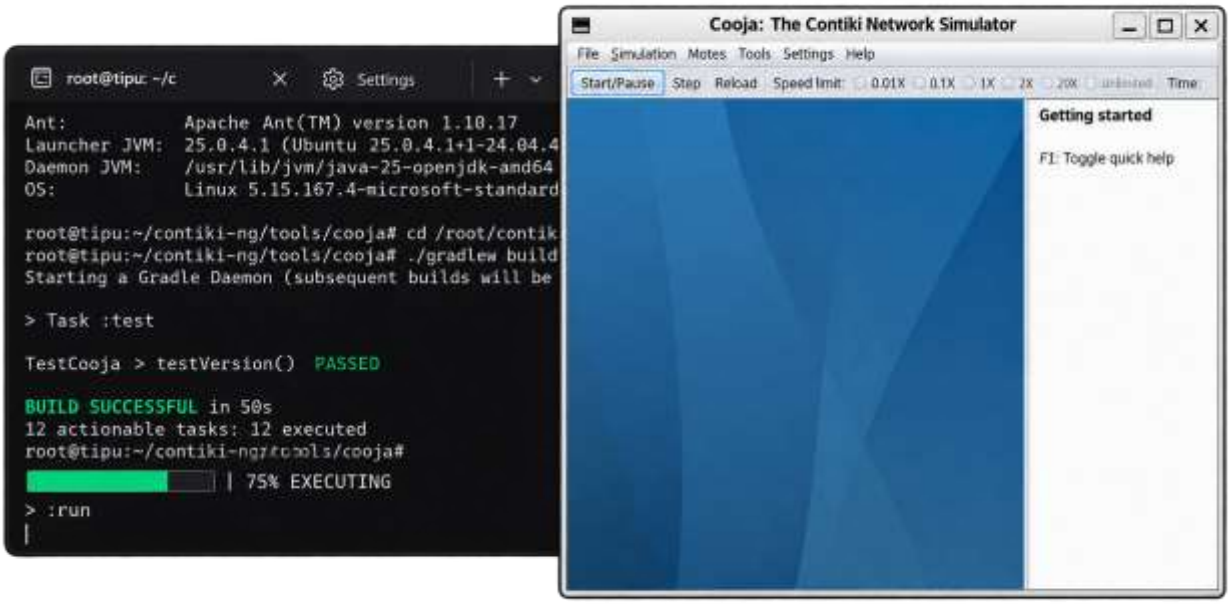
Both commands reported Java version 25.
step 7: Verify Gradle
The Cooja directory was opened using:
cd /root/contiki-ng/tools/cooja
The Gradle wrapper was executed:
./gradlew –version





This command runs the configured Gradle build tasks, including compilation and other checks specified by the project.
Step 9: Launch Cooja
After the build process, the simulator was launched using:
./gradlew run

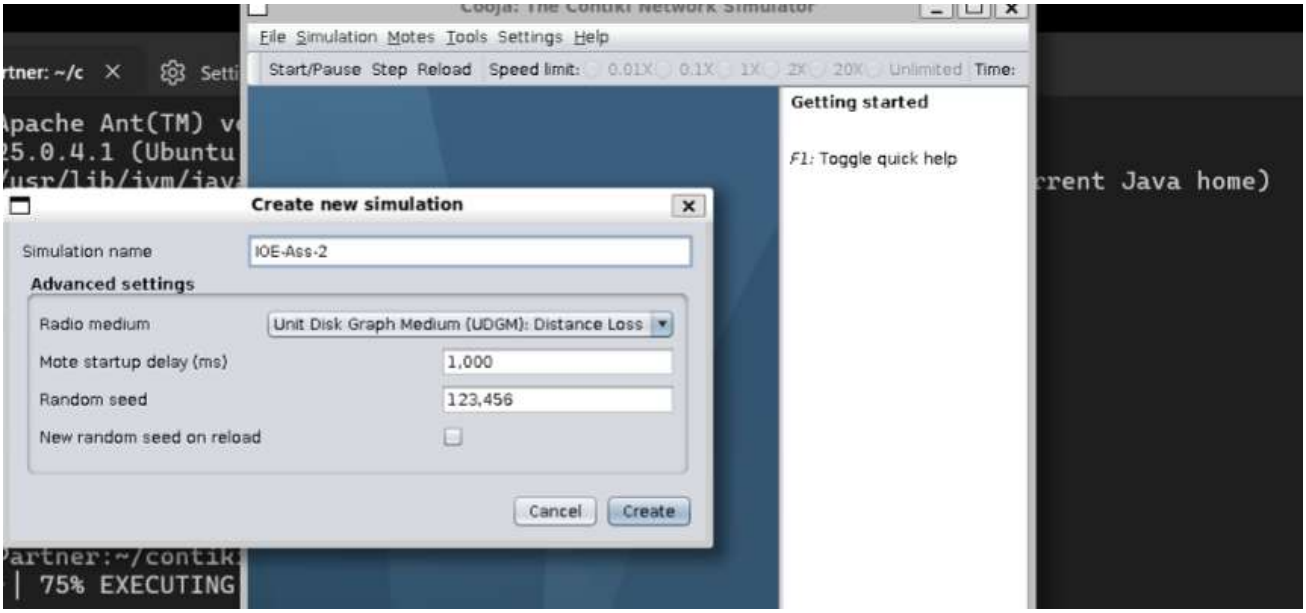


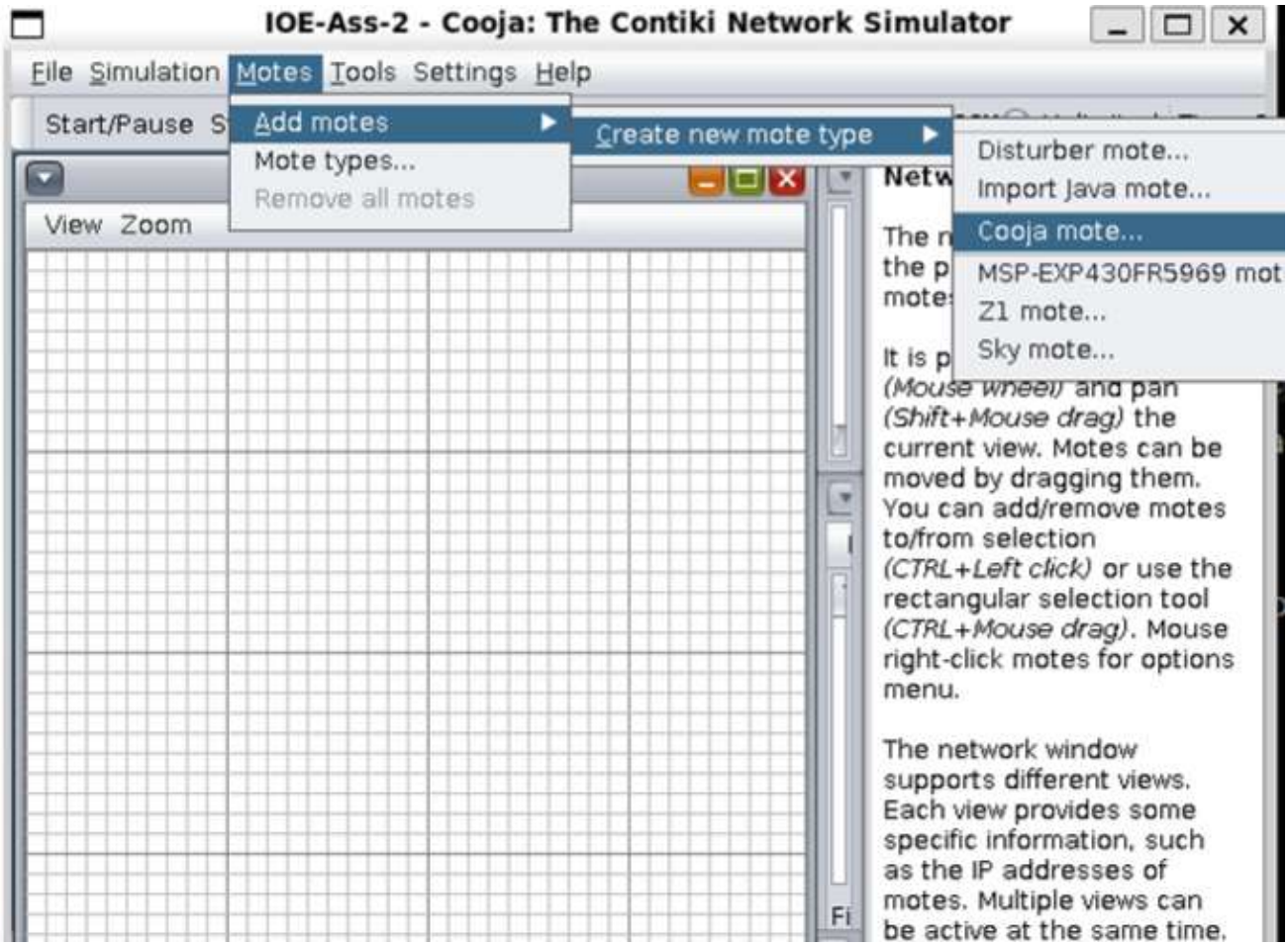


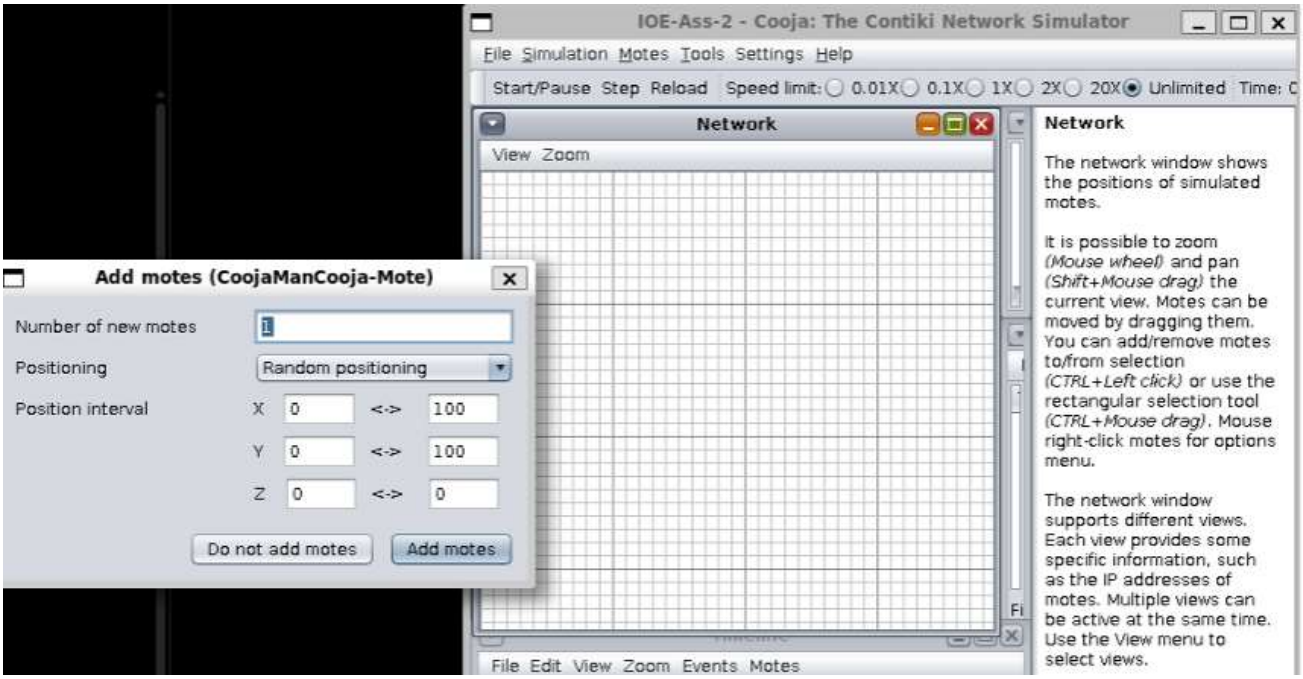
After launching Cooja, the following steps can be used to perform a basic simulation:

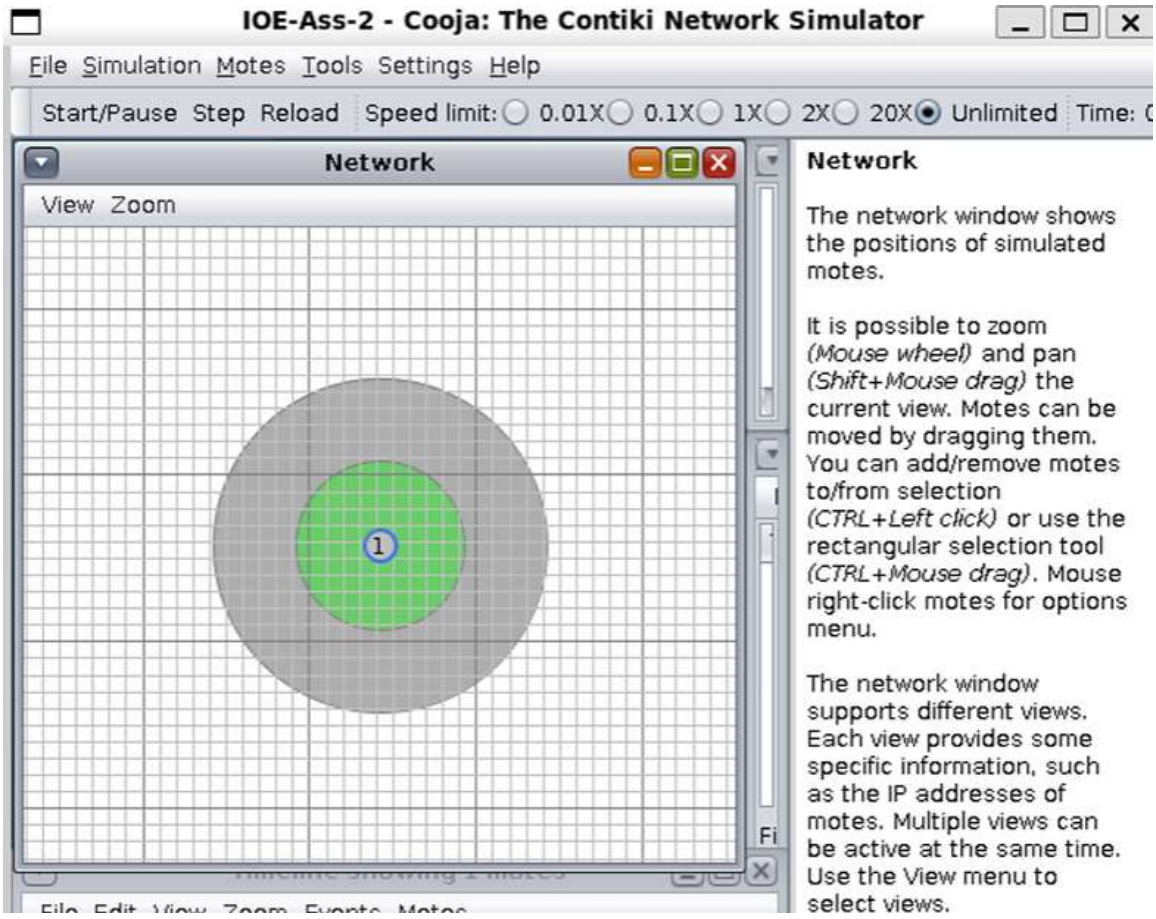
- Open the Cooja interface.
- Select File New Simulation.
- Enter a simulation name, such as Contiki First Simulation.
- Create a suitable mote type using the Motes menu.
- Select an appropriate Contiki-NG example application.
- Compile the application and create the required mote instances.
- Start the simulation using the Simulation Control panel.
- Observe the network topology and mote output.
- Save the simulation configuration as a .csc file.

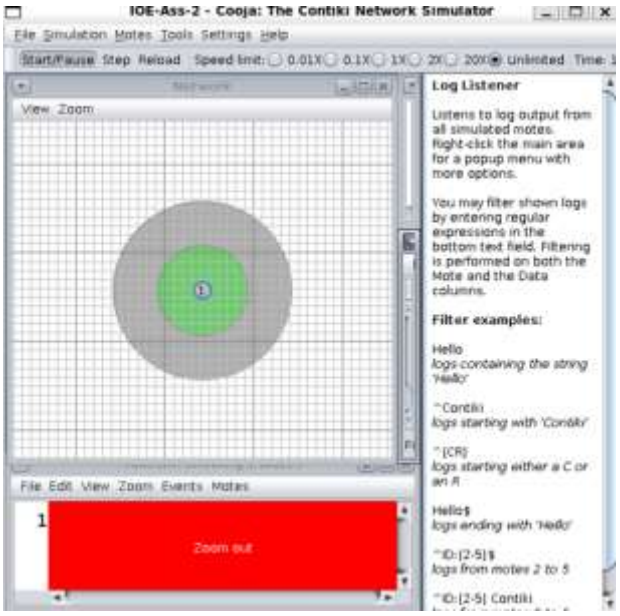
- A mote represents a virtual sensor node in the simulated network. Multiple motes can be
- added to study wireless communication and network behaviour.






IOE-Ass-2 - Cooja: The Contiki Network Simulator
File Simulation Motes Tools Settings Help
Start/Pause Step Reload Speed limit: 0.01X 0.1X 1X 2X 20X Unlimited Time:
Network
View Zoom
Network
The network window shows the positions of simulated motes.
It is possible to zoom (Mouse wheel) and pan (Shift+Mouse drag) the current view. Motes can be moved by dragging them. You can add/remove motes to/from selection (CTRL+Left click) or use the rectangular selection tool (CTRL+Mouse drag). Mouse right-click motes for options menu.
The network window supports different views. Each view provides some specific information, such as the IP addresses of motes. Multiple views can be active at the same time. Use the View menu to select views.
Add motes (CoojaManCooja-Mote)
Number of new motes
Positioning
Random positioning
Position interval
X 0 <-> 100
Y 0 <-> 100
Z 0 <-> 0
Do not add motes
Add motes
File Edit View Zoom Events Motes


IOE-Ass-2 - Cooja: The Contiki Network Simulator
File Simulation Motes Tools Settings Help
Start/Pause Step Reload Speed limit: 0.01X 0.1X 1X 2X 20X Unlimited Time:
Network
View Zoom
1
Network
The network window shows the positions of simulated motes.
It is possible to zoom (Mouse wheel) and pan (Shift+Mouse drag) the current view. Motes can be moved by dragging them. You can add/remove motes to/from selection (CTRL+Left click) or use the rectangular selection tool (CTRL+Mouse drag). Mouse right-click motes for options menu.
The network window supports different views. Each view provides some specific information, such as the IP addresses of motes. Multiple views can be active at the same time. Use the View menu to select views.



7. Observations

- The following observations were made during installation:
- Ubuntu 24.04.1 LTS was running successfully through WSL2.
- The Contiki-NG source repository was downloaded.
- The Cooja source files and Gradle wrapper were present in the downloaded
- repository.
- OpenJDK 25 was installed and verified.
- Gradle 9.6.1 executed successfully using Java 25.
- The Cooja graphical interface opened after executing ./gradlew run.

8. Applications

- Contiki-NG and Cooja are useful in the following areas:
- Internet of Things application development.
- Wireless sensor network simulation.
- Testing low-power communication protocols.
- Evaluating network topology and node communication.
- Prototyping embedded applications before hardware deployment.
- Studying the behaviour of constrained IoT devices.

9. Conclusion

The installation process of Contiki-NG and the Cooja Simulator was carried out using Ubuntu 24.04 LTS on WSL2. The necessary development tools, Java JDK 25, and Gradle were configured, and the Cooja graphical interface was launched successfully. This experiment demonstrates the basic setup required to develop and simulate IoT applications and wireless sensor networks using Contiki-NG and Cooja.